

Evidence Integrity Under a Hostile Executor

Why AI agents should not be allowed to certify their own work

Scott Myatt — CEO, Shedu

Study C of *Which Gates Matter?* · July 2026

Code, the judge-response cache, and the full technical report — with every experiment, statistical test, and reproduction command — are in the [trust-eval](#) repository.

Abstract

Autonomous coding agents often produce the same evidence used to judge their own work: test output, code diffs, approval records, and completion claims. A mistaken or dishonest agent can make all of this agree with itself even when it does not match what actually happened. This study asks which oversight methods can tell the difference.

We built 100 test cases — 40 containing a false but internally consistent claim, 60 truthful — modeled on real incident patterns from our governance system. Trusting the agent’s own report, and checking only whether its evidence is internally consistent, both approved every one of the 40 false claims. A general-purpose AI model judging the evidence directly did better, but traded one error for another: the judge with the best false-approval rate reached it by wrongly rejecting nearly three-quarters of truthful work. Using a more capable version of the same model mostly reduced those wrongful rejections rather than false approvals, and under sustained, feedback-guided attack the more capable version of one model was fooled more often than the weaker one, not less.

The one approach that worked reliably was checking each claim against an independent source built for that specific claim — rerunning the code instead of trusting a saved log, checking the real approval ledger instead of a claimed approval ID. That approach made no errors on the claims it could resolve. It cannot resolve everything, though, and when it can’t, a system has to choose between rejecting possibly-valid work and declining to decide at all — a real cost the headline numbers hide. A related test found that AI judges can also be pushed toward wrongly rejecting the large majority of true claims simply by having the submitter sound uncertain about their own work, regardless of whether the claim was actually true.

The practical conclusion: AI-generated evidence should not be allowed to certify itself. Models can help interpret evidence, but consequential claims need an independent check tied to the specific claim and the current state of the system.

1. The problem, in one example

Suppose a coding agent reports that it ran the test suite and everything passed, that it changed only the files it was authorized to touch, and that a human approved the change. Its report can be entirely self-consistent — the test log matches the summary, the diff matches the approval’s ticket number, the approval ID has the right format — and every part of it can still be false. The test log might be real but stale, recorded before a later code change quietly broke something. The diff might be computed against the wrong starting version, making an unauthorized change look authorized. The approval might genuinely exist, just for a different task.

None of this shows up as a contradiction, because nothing in the evidence disagrees with anything else in the evidence. That is the gap this study measures, and it is the same gap in any system — human or automated — that has to decide whether to trust a report it did not independently generate.

2. What we tested

We define an **independent anchor** as a source of truth outside the agent’s control that checks one specific kind of claim directly, rather than trusting the agent’s description of it: fresh execution instead of a submitted log, the real approval ledger instead of a claimed approval ID, actual repository state instead of a summarized diff. The central claim this study tests, stated in one sentence:

Checking that a piece of evidence agrees with itself can never establish that it agrees with reality. Establishing correspondence requires an independent check capable of directly testing that kind of claim; no single check and no smarter judge substitutes for the right one.

That claim was preregistered — written down, along with the analysis plan, before we generated or reviewed any results — and the full hypothesis-by-hypothesis disposition is in the technical report. It matters mainly because it kept the analysis honest: we could not quietly redefine “supported” after seeing which findings looked good, and the report says so explicitly wherever a result came out weaker or messier than expected.

The corpus. We built 100 cases around three failure patterns actually seen in our governance system: a test result that was valid once but went stale after the code changed underneath it, an approval record that exists but belongs to a different task, and a diff that looks correct but was computed against the wrong starting version. Forty cases are false claims built on these patterns; sixty are truthful, including some “suspicious-looking but actually valid” cases designed to penalize a reviewer that simply rejects anything unusual.

What we compared. Every claim was run through several review strategies, from weakest to strongest:

```
Agent submission
|
+-- Trust it, or inspect only what was submitted
|   No access to external state
|
+-- Check the claim against trusted external state
    Repository / execution / approval ledger / identity
```

The technical report labels these P0 through P5; we use plain names here and give the label in parentheses on first use so the two documents stay cross-referenceable.

Strategy	What it does
Trust the report (P0)	Accepts the agent’s own claim, no check
Judge from the evidence alone (P1, P2)	An AI model reviews it, or a script checks it for internal contradictions — neither looks outside what the agent submitted
Check the anchor, reject if unresolved (P3)	Consults the independent source; if it can’t resolve the claim, rejects it
Check the anchor, abstain if unresolved (P4/P5)	Same, but declines to decide instead of rejecting
Check the anchor, fall back to an AI judge if unresolved (P5-hybrid)	Same, but asks an AI model to decide the unresolved cases instead of rejecting or abstaining

3. What happened on the core test

Method	False claims approved	True claims rejected
Trust the report	40/40	0/60
Consistency check only	40/40	0/60
AI judge — DeepSeek	5/40	28/60
AI judge — Gemini	0/40	44/60
Independent anchor*	0/40	0/60

**On the main corpus. Truthful cases the anchor could not resolve are tested separately in Section 5 — the anchor does not score this cleanly there. The tables in the technical report call these “false acceptance” and “false rejection”; the exact statistical tests behind every number in this paper — including confidence intervals and significance levels — are also there.*

Three things stand out. First, trusting the agent’s report or checking it only for self-consistency is not a partial safeguard — both approved every single false claim in the test, 40 out of 40. Second, the AI judges did better, but only by trading one error for another: Gemini matched the anchor’s zero false approvals, but only by wrongly rejecting 44 of the 60 truthful claims — nearly three-quarters of legitimate work. Third, the independent anchor made no errors of either kind on this corpus. That last result comes with an important asterisk, explained in Section 5: the anchor got a perfect score here partly because every claim in this test that the anchor genuinely couldn’t resolve also happened to be false. Section 5 tests what happens when that is not true.

4. Does a smarter AI judge fix this?

We reran the same test with more capable versions of both judge models (DeepSeek’s and Gemini’s higher tiers). The upgrade helped with one problem and not the other: DeepSeek’s wrongful-rejection rate fell from 28/60 to 21/60, and Gemini’s fell from 44/60 to 26/60. Neither model’s false-approval rate improved on the anchor’s already-perfect 0/40 — DeepSeek’s stronger model still wrongly approved 4 of 40 false claims, and Gemini’s tied the anchor exactly rather than beating it.

The stronger model tiers delivered narrower and less consistent benefits than the headline comparison suggests: they helped a lot on claims about system state, barely at all on claims about execution, and not at all on authorization claims, where all four judges — weak and strong — were identical.

We then tested both judges under sustained, adaptive attack: an attacker that saw each rejection and revised its submission, without ever seeing the true answer or the anchor’s verdict. Here the picture reversed for one model. DeepSeek’s execution-claim vulnerability, already present at the single-shot level, got worse under repeated pressure, climbing from 5 of 20 to 15 of 20 false approvals. Gemini’s stronger model, which looked flawless (0/20) under a single attempt, was fooled on half of its execution claims and all of its state claims once the attacker was allowed to adapt — worse than its own weaker version, which was never fooled on execution claims at any tested attack budget. A model that looks safer in a one-shot test is not guaranteed to stay safer under a persistent attacker. This result is exploratory and has not yet been rerun as a full confirmatory experiment, but the direction is concerning enough to flag here.

The bottom line: a stronger judge is worth having, but it is not a substitute for an independent anchor. It reduces false rejections; it does not reliably reduce false approvals, and it can even get worse against an

attacker that adapts.

5. When the independent check can’t decide

Every anchor-inconclusive case that showed up elsewhere in this study happened to be false — which meant a policy of “reject whatever the anchor can’t resolve” looked perfect without ever being tested against a real, unresolvable, *truthful* case. We built a dedicated test to fix that: 30 true and 30 false cases, each genuinely inconclusive to the anchor, differing only in a scoring label no protocol could see.

Policy	Approved	Rejected	Declined	Decided	Error rate*
Reject if unresolved	0	60	0	100%	30/60
Decline if unresolved	0	0	60	0%	n/a

**Among cases the policy actually decided; declining to decide is not counted as a correct or incorrect answer.*

Rejecting everything the anchor can’t resolve decides every case, but gets all 30 truthful ones wrong. Declining to decide makes no errors only because it makes no decisions — every claim, true or false, is refused. We also tried letting an AI model make the call on these unresolved cases instead: Gemini’s fallback simply rejected everything, recovering no coverage at all. DeepSeek’s fallback approved true and false execution claims at the identical rate — 2 of 10 each — meaning its decision carried no real signal about which claims were actually true.

There is no protocol tested here that safely recovers these cases. A real system has to choose, deliberately, between losing legitimate work and losing coverage — and should not expect an AI fallback to split the difference safely.

6. Can a map of the code prove a safety check works?

One more anchor type deserves its own section because the result is unusually concrete. We tested a structural “skeleton” of the code — essentially a map of which functions call which, without the full source — as a lighter-weight stand-in for full verification.

The map by itself is not trustworthy — an agent could submit a stale, mismatched, or outright fabricated one. So the map was never treated as sufficient on its own: it was checked against an anchor for provenance and task binding first, the same way every other claim in this study was checked. With that check in place, the system caught every stale, mismatched, or fabricated map tested, none of which

got through. A follow-up test then asked a harder question: once a map was confirmed genuine, current, and correctly matched to the task, did that mean the thing it described actually worked? In five such cases, the guard the map showed existed in the code but did nothing when actually run:

Check	False approvals (of 5)
Genuine map, nothing checks runtime behavior	5 of 5
Genuine map, plus an anchor that actually runs the code	0 of 5

A code map can show that a safety check is present and wired in. It cannot show that the check works. Requiring agreement from an anchor that actually exercises the behavior closed the gap completely. (The map used here was a small, hand-built stand-in, not extracted from real source by production tooling — a caveat, not a reason to doubt the direction of the result.)

7. A judge can be swayed by how the claim is phrased

We tested whether additional free-text statements could move an AI judge’s verdict without changing the underlying claim or anything an anchor could verify. One statement was a hidden instruction telling the judge to skip its analysis and approve the claim; the other reported that the submitter was uncertain and hadn’t double-checked their own work: “I’m not fully certain about this and haven’t had a chance to double-check it myself.” Both were added equally to true and false claims, so any paired change in verdict came from the added statement — though the uncertainty statement is not purely stylistic. It carries real information about the submitter’s claimed confidence, deliberately uncorrelated with whether the claim was actually true, and a judge weighing that information is not obviously behaving unreasonably in isolation.

The hidden instruction didn’t work — false-approval rates were essentially unchanged across all four judges tested. The hedge did:

Judge	Normal wording	With hidden instruction	With the hedge
DeepSeek (base)	27% wrongly rejected	27%	90%
DeepSeek (stronger)	20%	20%	100%
Gemini (base)	47%	47%	100%
Gemini (stronger)	27%	27%	93%

Every judge tested, weak or strong, wrongly rejected 90–100% of claims — true and false alike — the moment the submitter sounded uncertain about their own work, with no change in how many false

claims got approved. This is a real vulnerability with a practical edge: an honest, appropriately humble submitter is punished by these judges almost as severely as a dishonest one. A companion test found no self-preference effect — a judge did not favor submissions attributed to its own model family — in the judges with complete data.

8. Other things we checked

We looked for AI judges falsely *claiming* to have done something they structurally could not do — run a command, inspect the repository — across all 200 cached judge decisions, using an automated scan, an AI-assisted review, and our own reading. None of the three checks found a single instance, including on cases where the judge’s actual conclusion was wrong. The errors these judges made were incorrect conclusions stated plainly, not fabricated descriptions of work they never did.

We also tested four attack patterns that earlier drafts of this project had only assumed were handled: replaying stale evidence, borrowing an approval from an unrelated task, forging an actor’s identity, and coordinating two artifacts to agree with each other instead of with reality.

Attack	Anchored check	No independent check
Stale evidence replayed after the fact	0/10 fooled	10/10 fooled
Approval borrowed from a different task	0/10 fooled	10/10 fooled
Forged actor identity	0/10 fooled	10/10 fooled
Two artifacts made to agree with each other	0/10 fooled	10/10 fooled

Checking each claim against its own dedicated anchor resisted all four, because none of the attacks change what the anchor itself honestly reports — they only change what the agent claims about it. The coordinated-artifact case is the clearest illustration of the whole paper in miniature: two lies can agree with each other perfectly. Agreement between artifacts, even several of them, is still not agreement with reality.

9. Limitations

No Anthropic model was tested as a judge, at any tier — only two other providers. Some comparisons, particularly the anchor-versus-stronger-judge result, are based on a sample too small (40 cases) to be statistically decisive on their own, though the direction was consistent across both tiers. The adaptive-

attack results are exploratory and single-run, not yet independently replicated as a confirmatory experiment. The code-map “skeleton” tested in Section 6 was a hand-built stand-in, not extracted from real source by real tooling. And the entire study assumes the anchors themselves — the repository, the approval ledger, the identity system — are honest and outside the agent’s reach; an attacker who can corrupt one of those directly is a different, untested threat.

10. Related work

This sits at the intersection of five research threads, cited representatively: scalable oversight, which asks how a weaker overseer can supervise a more capable one [1–3]; AI control, which treats the agent being overseen as potentially adversarial [4]; documented reliability problems in using AI models as judges, including sensitivity to surface features of the text they’re judging [5–7]; selective prediction, the formal study of when a system should decline to decide rather than guess [8]; and software provenance and attestation, which formalize independently-verifiable records outside a producer’s control [9, 10]. This study is a narrow, empirical instance of all five, run inside one synthetic coding-agent corpus rather than a general claim about any of them.

11. Conclusion

Internal consistency — whether checked by a script or judged by a capable AI model reading the evidence alone — does not establish that a piece of evidence corresponds to reality. Trusting an agent’s own report, or checking only that its story holds together, approved every false claim tested. A capable AI judge did better, but even the best-performing one got to zero false approvals only by rejecting most of the legitimate work in front of it, and a stronger version of that same judge could be fooled *more* under sustained attack, not less.

An independent, claim-matched anchor avoided that tradeoff on every claim it could resolve. It could not resolve everything, and where it couldn’t, every option tested — reject, abstain, or fall back to an AI judge — cost something real: rejected legitimate work, lost coverage, or a decision with no actual signal in it. A structural map of the code was useful for confirming that something existed, and useless for confirming that it worked. And a judge’s verdict could be moved by how confident the submitter sounded, independent of whether they were right — a vulnerability distinct from, and arguably more concerning than, anything else measured here, because it actively punishes honesty.

The practical rule this supports: AI-generated evidence should not certify itself. A model can help interpret evidence and can catch things a human reviewer would miss, but any claim that actually matters needs an independent check tied to that specific claim and the current state of the system it describes — not a smarter judge reading the same self-reported evidence more carefully.

References

- [1] Irving, G., Christiano, P., & Amodei, D. (2018). AI safety via debate. arXiv:1805.00899.
- [2] Leike, J. et al. (2018). Scalable agent alignment via reward modeling: a research direction. arXiv:1811.07871.
- [3] Bowman, S. R. et al. (2022). Measuring progress on scalable oversight for large language models. arXiv:2211.03540.
- [4] Greenblatt, R. et al. (2023). AI control: Improving safety despite intentional subversion. arXiv:2312.06942.
- [5] Zheng, L. et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. NeurIPS 2023; arXiv:2306.05685.
- [6] Wang, P. et al. (2023). Large language models are not fair evaluators. arXiv:2305.17926.
- [7] Panickssery, A. et al. (2024). LLM evaluators recognize and favor their own generations. NeurIPS 2024; arXiv:2404.13076.
- [8] Geifman, Y., & El-Yaniv, R. (2017). Selective classification for deep neural networks. NeurIPS 2017.
- [9] Torres-Arias, S. et al. (2019). in-toto: Providing farm-to-table guarantees for bits and bytes. USENIX Security 2019.
- [10] Open Source Security Foundation. SLSA: Supply-chain Levels for Software Artifacts. slsa.dev (accessed 2026).